AN INTERPRETABLE MULTI-MODAL SPAM DETECTION FRAMEWORK USING EXPLAINABLE AI AND MACHINE LEARNING

A PROJECT REPORT

Submitted by

Aariz (23BCS10422)
Naman (23BCS11036)
Vanshit Kumar (23BCS10087)
Aditi Arya (23BCS11535)
Avish (23BCS13898)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Chandigarh University

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report **"AN INTERPRETABLE MULTI-MODAL SPAM DETECTION FRAMEWORK USING EXPLAINABLE AI AND MACHINE LEARNING"** is the bonafide work of **"Aariz (23BCS10422), Naman (23BCS11036), Vanshit Kumar (23BCS10087), Aditi Arya (23BCS11535) and Avish (23BCS13898)"** who carried out the project work under my/our supervision.

SIGNATURE

Dr. Surinder Kaur

HEAD OF THE DEPARTMENT

Department of Computer Science & Engineering

SIGNATURE

Sonam Juneja (E11002)

SUPERVISOR

Assistant Professor

Department of Computer Science & Engineering

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, Sonam Juneja (E11002), Assistant Professor, Department of Computer Science & Engineering, Chandigarh University, for the continuous guidance, constructive criticism and encouragement extended throughout the course of this project. Her insistence that every design decision be justified — and every prediction be explainable — shaped the central theme of this work.

We are equally thankful to Dr. Surinder Kaur, Head of the Department, for providing the computing facilities and the academic environment that made this work possible. Our thanks also go to the faculty members of the department for their timely feedback during the project reviews.

Finally, we thank our families and friends for their constant support and motivation.

Aariz (23BCS10422)
Naman (23BCS11036)
Vanshit Kumar (23BCS10087)
Aditi Arya (23BCS11535)
Avish (23BCS13898)

TABLE OF CONTENTS

List of Figures i

List of Tables ii

ABSTRACT iv

GRAPHICAL ABSTRACT v

Chapter 1. INTRODUCTION 1

1.1 Client Identification/Need Identification/Identification of relevant Contemporary issue 1

1.2 Identification of Problem 2

1.3 Identification of Tasks 2

1.4 Timeline 3

1.5 Organization of the Report 3

Chapter 2. LITERATURE REVIEW/BACKGROUND STUDY 4

2.1 Timeline of the reported problem 4

2.2 Proposed solutions 4

2.2.1 Rule-based and collaborative filtering 4

2.2.2 Classical statistical learning 4

2.2.3 Deep learning 5

2.2.4 Explainable-AI methods 5

2.2.5 Recent advances (2024–2026) 5

2.3 Bibliometric analysis 6

2.4 Review Summary 7

2.5 Problem Definition 9

2.6 Goals/Objectives 9

Chapter 3. DESIGN FLOW/PROCESS 10

3.1 Evaluation & Selection of Specifications/Features 10

3.2 Design Constraints 10

3.3 Analysis and Feature finalization subject to constraints 11
3.4 Design Flow 12
3.4.1 Design I: Fine-tuned transformer classifier 12
3.4.2 Design II: Hybrid classical ensemble with a dual XAI layer 12
3.5 Design selection 13
3.6 Implementation plan/methodology 14
3.6.1 Hyper-parameter configuration 15
3.6.2 Training and inference procedures 16
3.6.3 Modality router 16
Chapter 4. RESULTS ANALYSIS AND VALIDATION 17
4.1 Implementation of solution 17
4.1.1 Experimental setup 18
4.1.2 Classification results 19
4.1.3 Statistical significance of the differences 22
4.1.4 Ensemble behaviour and error analysis 23
4.1.5 Routing and cross-channel transfer 24
4.1.6 Explainability results 25
4.1.7 Computational cost 30
4.1.8 Validation on realistic unseen messages 30
4.1.9 Interpretation and data validation 32
Chapter 5. CONCLUSION AND FUTURE WORK 34
5.1 Conclusion 34
5.2 Future work 34
APPENDIX 36

List of Figures

Figure 1.1: Gantt chart of the project schedule 3

Figure 3.1: Block diagram of the proposed interpretable multi-modal detector (Design II) 13

Figure 3.2: Implementation flowchart of the training and evaluation pipeline . 15

Figure 4.1: Detector tab of the Streamlit application after analysing a message 17

Figure 4.2: Class balance per modality and message-length distributions 18

Figure 4.3: Test-set metrics per model and modality (vertical axis truncated at 0.90 to resolve the differences) 20

Figure 4.4: ROC curves for all four systems: SMS (left) and e-mail (right) . . . 20

Figure 4.5: Test-set accuracy, precision, recall and F1 of the four systems: SMS (top) and e-mail (bottom) 21

Figure 4.6: Confusion matrices of the hybrid ensembles on the held-out test sets 23

Figure 4.7: Confusion matrices of all four systems on the held-out test sets: SMS (left) and e-mail (right) 24

Figure 4.8: Local explanations of the exhibited SMS: LIME word attributions over the full ensemble (left) and SHAP waterfall of the XGBoost component (right) 26

Figure 4.9: LIME explanations of two held-out spam messages over the full ensemble: SMS (left) and e-mail (right); red bars push towards spam, blue towards ham 26

Figure 4.10: Global SHAP beeswarm summaries: SMS (left) and e-mail (right); UPPER-CASE rows are engineered stylometric features, lower-case rows are words 28

Figure 4.11: Global feature importance (mean |SHAP|, exact TreeSHAP on the XGBoost component, 800 held-out messages): SMS (left) and e-mail (right) 28

List of Tables

Table 2.1: Comparison of reported spam-detection approaches 7

Table 2.2: Recent studies (2024–2026) related to this project 8

Table 3.1: Normalisation rules applied before TF–IDF vectorisation (in order) 11

Table 3.2: Comparison and selection of the design alternatives 14

Table 3.3: Hyper-parameters and experimental settings 15

Table 4.1: Corpora used for training and evaluation (after cleaning) 18

Table 4.2: SMS — held-out test set (1,032 messages) 19

Table 4.3: E-mail — held-out test set (2,400 messages) 20

Table 4.4: Additional metrics on the held-out test sets (lower Brier score and FPR are better) 22

Table 4.5: Hybrid ensemble: point estimates and 95% bootstrap confidence intervals (2,000 resamples of the test set) 22

Table 4.6: Paired significance tests of the hybrid ensemble against each base learner (b / c: messages that only the ensemble / only the base learner classifies correctly; p-values Holm-corrected) 23

Table 4.7: Representative misclassified test messages (ensemble spam probability; threshold 0.5) 25

Table 4.8: Routing and cross-channel transfer of the ensembles (pooled test set of 3,432 messages) 25

Table 4.9: Deletion test of LIME faithfulness: mean drop in the ensemble spam probability after removing the top-k LIME words / k random words 27

Table 4.10: SMS: the ten most important features by mean |SHAP| (exact TreeSHAP, XGBoost component, 800 held-out messages); the last column shows where high values of the feature push the prediction . 29

Table 4.11: E-mail: the ten most important features by mean |SHAP| (exact TreeSHAP, XGBoost component, 800 held-out messages); the last column shows where high values of the feature push the prediction . 29

Table 4.12: Shortcut probe on the e-mail test set: appending the word "enron" to detected spam, and deleting it from legitimate mail ($\overline{\Delta p}$: mean change in spam probability) 30

Table 4.13: Computational cost on a 10-core ARM64 CPU without a GPU (median times) 30

Table 4.14: Validation on realistic messages written for this project (not taken from either corpus); spam probability from the routed ensemble . . 31

Table 4.15: Automated end-to-end smoke tests (final run: all checks passed) . . . 33

ABSTRACT

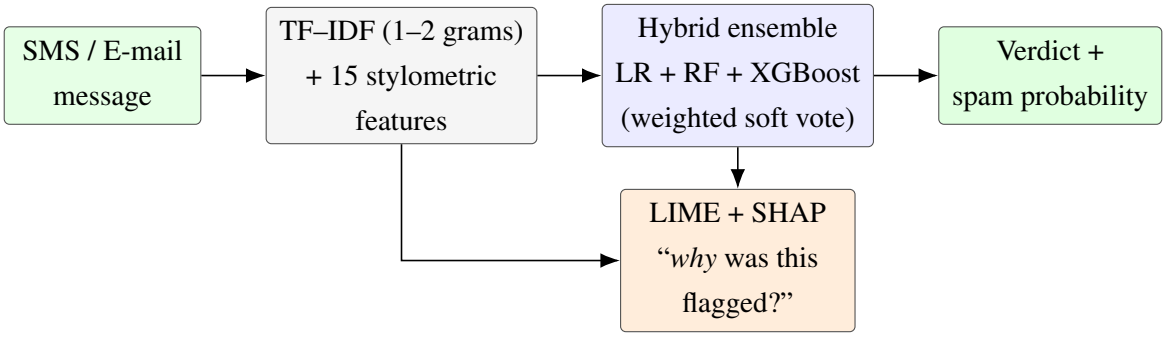
Fraudulent messages now reach users through more than one channel, and the machine-learning filters that intercept them are usually black boxes: they return a verdict but no reason. Aggregate accuracy alone can conceal systematic errors and dataset artefacts, leaves the end user with no basis to contest a decision, and gives an analyst nothing to audit.

This project presents an interpretable multi-modal spam detection framework covering both SMS and e-mail. Each modality is served by its own specialist pipeline: text normalisation that maps high-signal entities to readable placeholder tokens (`httpaddr`, `numbr`, `moneysymb`, `phonenumbr`), TF–IDF features over uni- and bi-grams combined with fifteen engineered stylometric statistics, and a hybrid ensemble of Logistic Regression, Random Forest and XGBoost fused by performance-weighted soft voting. A lightweight router selects the appropriate specialist at inference time. A dual explanation layer justifies every prediction: LIME explains the complete ensemble locally, while exact TreeSHAP provides local and global attributions for the boosted-tree component. The framework ships as a fully reproducible Python pipeline and a four-tab interactive Streamlit application.

Evaluated on 17,158 messages from two public research corpora using an 80/20 stratified split and 5-fold cross-validation, the hybrid ensemble achieved 98.55% accuracy (ROC-AUC 0.9941) on the UCI SMS Spam Collection and 98.25% accuracy (ROC-AUC 0.9983) on the Enron-Spam corpus, matching (to four decimal places) or beating every base learner on ROC-AUC. Beyond the numbers, the explanation layer exposed a genuine dataset bias — the token `enron` acting as a shortcut for legitimate mail — that no accuracy table could reveal, demonstrating the practical value of shipping explanations together with predictions.

Keywords: spam detection, explainable AI, ensemble learning, SHAP, LIME, TF–IDF, multi-modal classification, machine learning.

GRAPHICAL ABSTRACT



Every message is routed to a modality-specific feature space, scored by a hybrid ensemble of three heterogeneous learners, and returned to the user together with a human-readable explanation of the verdict.

CHAPTER 1.

INTRODUCTION

1.1. Client Identification/Need Identification/Identification of relevant Contemporary issue

Unsolicited and fraudulent messaging is a documented, growing contemporary problem. The FBI's Internet Crime Complaint Center recorded close to 3,00,000 phishing/spoofing complaints in calendar year 2023 — the single largest crime category in its annual report — and industry telemetry has for years placed unsolicited traffic at roughly half of all e-mail exchanged worldwide. The problem is equally visible on the SMS channel: as banking, commerce and one-time-password flows moved to text messaging, so did the scams ("smishing"). In India, the volume of unsolicited commercial communication led the Telecom Regulatory Authority of India to notify the Telecom Commercial Communications Customer Preference Regulations, 2018, and to mandate a distributed-ledger registration system for all bulk senders — regulatory evidence that the issue is both real and unresolved.

The most recent edition of the same report, covering 2025, shows the problem still growing: the Internet Crime Complaint Center received more than one million complaints with reported losses of US$20.9 billion, phishing and spoofing was again the most reported category (191,561 complaints), and 22,364 complaints involved artificial intelligence (Federal Bureau of Investigation, 2026). Generative AI is changing the character of the threat as well as its volume: large language models can now write persuasive, personalised phishing at negligible cost (Heiding et al., 2024; Yao et al., 2024), and phishing produced in this way has been shown to get past the filters of major webmail providers (Opara et al., 2025).

Filters exist, but a second, newer need has emerged around *transparency*. Modern spam classifiers are statistical black boxes: a message disappears and nobody — not the end user, not the platform analyst, not the auditor — can say why. Transparency obligations in data-protection law (Article 22 of the GDPR in Europe; the Digital Personal Data Protection Act, 2023 in India) are pushing automated decision systems toward explainability, and a filter that cannot justify its verdicts is increasingly difficult to defend.

The client for this work is any operator of a user-facing messaging channel — typified by a university IT cell that receives phishing-report tickets from students and staff — who needs a filter that is (a) accurate across more than one message channel, (b) able to attach a human-readable justification to every verdict so that flagged messages can be triaged and contested, and (c) cheap enough to train and audit on commodity hardware. This is a

consultancy-style need: the client does not merely want a spam score, they want a *defensible* spam score.

1.2. Identification of Problem

No openly available framework simultaneously (i) handles both SMS and e-mail, whose statistics differ radically (median message length of 12 words versus 155; spam prevalence of about 12% versus about 48% in the public research corpora), (ii) matches the state-of-the-art accuracy of classical machine-learning filters, and (iii) attaches a faithful, per-prediction explanation to every verdict. The consequences of this gap are concrete: verdicts cannot be verified by non-experts, misclassified legitimate messages cannot be contested, and hidden dataset biases pass silently into deployed models. The problem addressed by this project is therefore the design, implementation and validation of an accurate, fully interpretable, multi-modal spam detection framework.

1.3. Identification of Tasks

The work was decomposed into the following tasks:

1. Survey of the literature on spam/scam detection and on post-hoc explainable-AI techniques.
2. Acquisition, cleaning and de-duplication of two public corpora (UCI SMS Spam Collection; Enron-Spam), followed by exploratory data analysis.
3. Design of an interpretability-preserving preprocessing scheme and feature space (readable placeholder tokens; engineered stylometric features).
4. Training and tuning of three heterogeneous base learners per modality with class-imbalance handling.
5. Design and implementation of the performance-weighted soft-voting ensemble and the modality router.
6. Integration of the explanation layer (LIME for the full ensemble; exact TreeSHAP for the boosted-tree component) at local and global scope.
7. Development of the four-tab interactive Streamlit demonstration application and an automated end-to-end smoke-test suite.
8. Evaluation on held-out test sets, error and bias analysis, and documentation of the results in this report.

1.4. Timeline

The project was executed over one semester of approximately fifteen working weeks. The schedule is represented as a Gantt chart in Figure 1.1; the model-training, explanation-layer and application workstreams deliberately overlap between weeks 8 and 12 because the explanation artefacts were used to debug the models as they were being trained.

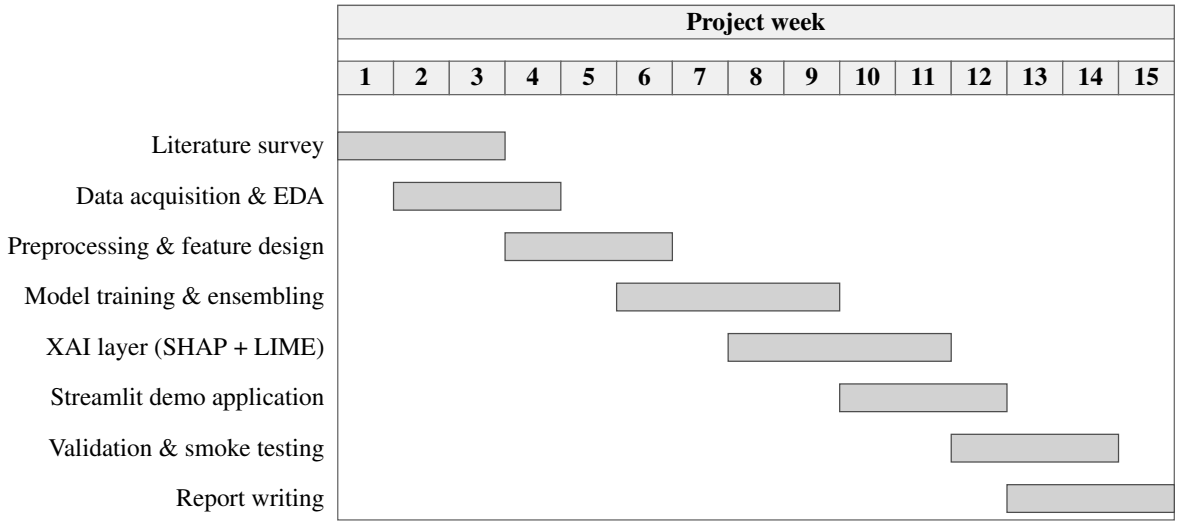


Figure 1.1: Gantt chart of the project schedule

1.5. Organization of the Report

Chapter 2 reviews the reported literature on spam detection and explainable AI and closes with the formal problem definition and objectives. Chapter 3 covers the evaluation of candidate features, the design constraints, the two alternative design flows that were considered and the justification for the selected design. Chapter 4 presents the implementation using modern engineering tools together with the complete experimental results, error analysis and explainability findings. Chapter 5 concludes the work and identifies the way ahead. The appendices list the software environment, the complete engineered feature set, the derived ensemble weights and the scam-keyword lexicon.

CHAPTER 2.

LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

Unsolicited electronic messaging is almost as old as electronic messaging itself: the first documented mass advertisement was sent over ARPANET in 1978. Volumes exploded with commercial e-mail in the 1990s, and the first generation of defences was rule-based — keyword blacklists, sender blocklists and hand-written heuristics. The statistical era opened when Sahami et al. (1998) framed junk-mail filtering as Naive-Bayes text classification, followed quickly by margin-based methods (Drucker et al., 1999) and, through the 2000s, by tree ensembles such as Random Forests (Breiman, 2001). Research on the SMS channel became reproducible when Almeida and Gómez Hidalgo (2011) released the UCI SMS Spam Collection, while the Enron-Spam corpora (Metsis et al., 2006) did the same for personalised e-mail filtering. From roughly 2015, deep neural models pushed benchmark accuracy still higher but at a severe cost in transparency — which in turn triggered the explainable-AI wave: LIME (Ribeiro et al., 2016) and SHAP (Lundberg and Lee, 2017) made post-hoc, per-prediction explanation practical. Most recently, regulation (GDPR, 2018; DPDP Act, 2023) has begun to treat explainability as a compliance requirement rather than a luxury, which is the point in the timeline where this project positions itself.

2.2. Proposed solutions

Reported approaches fall into four broad families.

2.2.1. Rule-based and collaborative filtering

Blacklists, keyword rules and community-scored heuristic engines (e.g. SpamAssassin-style scoring). They are fully transparent and cheap, but brittle: spammers evade fixed rules within days, false-positive rates are high, and the rule bases require continuous manual curation.

2.2.2. Classical statistical learning

Naive Bayes, logistic regression, support vector machines and tree ensembles trained on bag-of-words or TF–IDF representations. These methods reach 95–99% accuracy on the public benchmark corpora, train in seconds to minutes on commodity CPUs, and expose at least

coarse interpretability (coefficients, feature importances). Class imbalance — severe on the SMS channel — must be handled explicitly.

2.2.3. Deep learning

Convolutional, recurrent and, more recently, transformer-based text classifiers. They deliver the highest raw benchmark numbers but demand GPUs and large training sets, and their decision process is opaque; attention weights are known to be an unreliable proxy for faithful explanation.

2.2.4. Explainable-AI methods

Post-hoc techniques that explain an already-trained classifier. LIME fits a local linear surrogate around one prediction of *any* model by perturbing the input; SHAP computes additive Shapley-value attributions, with an exact polynomial-time algorithm (TreeSHAP) for tree ensembles. Combining a strong classical ensemble with these two techniques promises near state-of-the-art accuracy *with* faithful explanations.

2.2.5. Recent advances (2024–2026)

Research on message filtering has moved quickly in the last three years. The studies most relevant to this project are summarised below by theme.

SMS spam and smishing. Deep and transformer models dominate recent SMS work. Mehmood et al. (2024) designed a deep-learning smishing detector aimed explicitly at reducing false positives, and Uddin et al. (2025) combined a transformer language model with an explainability analysis. Prince et al. (2026) proposed an explainable hybrid transformer that fuses several feature types for smishing detection, while Jasim et al. (2026) paired fuzzy logic, which uses Gaussian membership functions to capture linguistic uncertainty, with a bidirectional LSTM to make SMS filtering transparent. Hur et al. (2026) benchmarked large language models (LLMs) on English and Korean SMS datasets with zero-shot, few-shot and chain-of-thought prompting and found them robust under concept drift and adversarial attack. Two further findings are especially relevant here. Şahin et al. (2026) evaluated 42 combinations of seven text representations and six classical classifiers on the same SMS corpus and concluded that the feature representation matters more than classifier complexity, with character 3-grams and Logistic Regression reaching 98.55% accuracy. Salman et al. (2024) released a new SMS dataset and analysed the evasive techniques that spammers use against filters, and Mambina et al. (2024) extended deep SMS detection to Swahili.

E-mail spam and phishing. Fine-tuned transformers and LLMs are now the most

common e-mail detectors: Jamal et al. (2024) built an improved transformer for three-way phishing/spam/ham classification, Roumeliotis et al. (2024) compared fine-tuned LLMs, NLP models and CNNs, Uddin et al. (2026) explained a transformer phishing detector, and Koide et al. (2026) used an LLM to detect phishing and explain its verdicts to end users. The margin over classical methods is small: Meléndez et al. (2024) found that RoBERTa reached 0.9943 accuracy on phishing e-mail against 0.9876 for the best traditional model, a support vector machine. Ensembles remain competitive: Kshirsagar et al. (2025) reported 0.9905 accuracy and 0.9991 ROC-AUC with a meta-learner on Enron-Spam combined with TREC 2007, and Hadi et al. (2026) fused shallow lexical features with deep contextual embeddings in a hybrid aimed at both channels, which they evaluated on the SMS Spam Collection (98.8% accuracy, ROC-AUC 0.993). On the attack side, Heiding et al. (2024) showed that GPT-4 can generate phishing e-mails automatically, and Opara et al. (2025) found that AI-generated phishing got past the filters of major webmail providers but could be detected with 60 stylometric features and XGBoost (96% accuracy) — direct support for the stylometric features used in this project.

Explainability in message filtering. Shambour et al. (2025) combined metaheuristic feature selection with SHAP explanations for spam detection, and Panthi et al. (2026) compared SHAP and LIME on three boosting models for phishing URLs, finding SHAP's attributions more stable at the global level and LIME's more instance-specific. Several authors warn that explanations cannot be taken at face value. Lyu et al. (2024) reviewed more than 110 explanation methods in NLP through the lens of faithfulness, Krishna et al. (2024) showed that different explanation methods frequently disagree and that practitioners lack a principled way to resolve the disagreement, and Longo et al. (2024) set out the open challenges of the field.

Shortcut learning. A model can reach high accuracy by exploiting spurious cues that separate the classes in one dataset instead of learning the intended concept. Du et al. (2024) documented such shortcuts in large language models, and Dogra et al. (2024) surveyed how explanation methods can expose the dataset biases behind them. Spam corpora are particularly exposed because their legitimate messages often come from a single organisation — precisely the situation this project later uncovered in the Enron-Spam corpus.

2.3. Bibliometric analysis

A comparison of the reviewed families on key features, effectiveness and drawbacks is summarised in Table 2.1.

Table 2.1: Comparison of reported spam-detection approaches

Approach	Key feature	Effectiveness	Drawback
Rule-based filters	Hand-written, fully transparent rules	Moderate; high false positives	Easily evaded; constant manual updating
Naive Bayes	Fast probabilistic word model	95–98% on benchmark corpora	Feature-independence assumption
SVM / Logistic Regression	Margin / linear decision boundary	97–98%	Linear view of language; needs calibration
Tree ensembles (RF, XGBoost)	Non-linear feature interactions	97–99%	Interpretability limited to importances
Deep / transformer models	Learned contextual representations	98–99%+	Opaque; GPU and data hungry
Classical ensemble + XAI (this work)	Accuracy of ensembles with per-verdict explanations	98.3–98.6% (measured)	Explanation adds inference cost

Table 2.2 places the recent studies reviewed above next to this project. Their figures come from different datasets and protocols, so they indicate the state of the art rather than allowing a strict ranking.

2.4. Review Summary

The literature exhibits a persistent trade-off: the most transparent systems are the least accurate, and the most accurate are the least transparent. Furthermore, nearly all published work targets a single channel, even though SMS and e-mail scams are two faces of the same problem with radically different message statistics. A hybrid ensemble of heterogeneous classical learners, wrapped in a dual model-agnostic/model-specific explanation layer and specialised per modality, occupies the unclaimed middle ground: benchmark accuracy within a point of deep models, full per-prediction transparency, and CPU-only training. This project adopts exactly that position.

The recent literature sharpens three further gaps. First, in the studies reviewed here explanations

Table 2.2: Recent studies (2024–2026) related to this project

Study	Channel	Approach	Explanation	Reported result
Şahin et al. (2026)	SMS	42 pairs of classical representations and classifiers	–	98.55% accuracy (character 3-grams + LR)
Hadi et al. (2026)	SMS (e-mail in scope)	Deep–shallow hybrid	–	98.8% accuracy, ROC-AUC 0.993
Hur et al. (2026)	SMS (English, Korean)	LLM prompting	LLM rationales	Robust under drift and attack
Meléndez et al. (2024)	E-mail (phishing)	Transformers vs. traditional ML	–	RoBERTa 0.9943 vs. SVM 0.9876 accuracy
Kshirsagar et al. (2025)	E-mail	Meta-learner ensemble	Interpretable models	0.9905 accuracy, 0.9991 ROC-AUC
Opara et al. (2025)	E-mail (phishing written by AI)	60 stylometric features + XGBoost	–	96% accuracy
This project	SMS + e-mail (routed)	TF–IDF + stylometry; weighted soft vote of LR, RF, XGBoost	LIME + exact TreeSHAP	98.55% / 98.25% accuracy; ROC-AUC 0.9941 / 0.9983

are mostly presented as illustrations of single predictions, and their faithfulness is rarely tested, although the XAI literature shows that it cannot be assumed. Second, high benchmark accuracy is seldom examined for shortcuts, even though organisation-specific corpora such as Enron-Spam invite them. Third, model comparisons on near-saturated benchmarks seldom report confidence intervals or significance tests, so differences in the third decimal place are hard to interpret. This project addresses all three in addition to the two-channel design.

2.5. Problem Definition

To design, build and validate an interpretable multi-modal spam detection framework that classifies both SMS and e-mail messages as legitimate or fraudulent using a hybrid ensemble of Logistic Regression, Random Forest and XGBoost, and that attaches a faithful local explanation (LIME over the full ensemble; SHAP over the boosted-tree component) to every verdict, together with global model summaries. The framework must be reproducible end-to-end on a commodity CPU and be operable by a non-expert through an interactive application. The work explicitly excludes non-English corpora, adversarially obfuscated spelling, image-based spam, and hardening for production deployment.

2.6. Goals/Objectives

- Achieve at least 97% test-set accuracy on both public corpora (UCI SMS Spam Collection; Enron-Spam).
- Ensure the hybrid ensemble matches or beats every base learner on ROC-AUC for both modalities.
- Attach a local explanation to every single prediction and provide global explanations of what each model has learned.
- Keep the complete pipeline — data download, training, evaluation, figure generation — reproducible on a commodity CPU in under five minutes, verified by an automated test suite.
- Deliver an interactive demonstration application usable by a non-expert.

CHAPTER 3.

DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

The candidate features identified in the literature were critically evaluated against the client need of *accurate and auditable* filtering.

Retained. Modality-specific pipelines (one specialist per channel, selected by a lightweight, user-overridable router); normalisation of high-signal entities into *readable* placeholder tokens (`httpaddr`, `numbr`, `moneysymb`, `phonenumbr`) instead of deleting them; fifteen engineered stylometric features computed on the raw message and named in UPPER-CASE so that they remain visually distinct from vocabulary in every explanation plot; TF–IDF over uni- and bi-grams with sublinear term frequency; a heterogeneous three-model ensemble (linear / bagging / boosting) fused by performance-weighted soft voting; balanced class weights for imbalance handling; and a dual explanation layer (LIME + SHAP).

Rejected. Stemming and lemmatisation (they destroy the surface word forms on which readable explanations depend); deletion of URLs, numbers and currency symbols during cleaning (they are among the strongest scam signals); deep contextual models as the primary classifier (opaque, GPU-dependent); synthetic minority oversampling of text (it manufactures training "messages" that never existed and cannot be meaningfully explained); and an on-premise mail-server integration, which added engineering cost without addressing the stated research need.

The retained normalisation step is specified in Table 3.1. The rules are applied in the order listed, after the text has been lower-cased, so that, for example, a phone number is recognised before its digits could be replaced by the generic number token.

3.2. Design Constraints

- **Regulations:** transparency expectations of GDPR Article 22 and the DPDP Act, 2023 motivate per-prediction explanations; the research licences of both corpora restrict the system to research and education use, and it is labelled accordingly.
- **Economic:** zero licence cost (fully open-source stack) and commodity-CPU training; no GPU may be assumed.
- **Environmental:** CPU-only training completing in about two minutes keeps the energy footprint negligible compared with GPU-trained deep models.

Table 3.1: Normalisation rules applied before TF–IDF vectorisation (in order)

Step	Pattern	Replaced by	Reason
1	HTML tags	a space	Markup carries no meaning for the reader
2	URLs (`http(s)://...`, `www....`, or a word ending in `.com`, `.net`, `.org`, `.info`, `.co.uk`)	`httpaddr`	Links are a primary scam signal
3	E-mail addresses	`emailaddr`	Contact details in unsolicited messages
4	A currency symbol (dollar, pound, euro, rupee) followed by an amount, or an amount followed by a currency word (dollars, pounds, euros, rs, ...)	`moneysymb`	Prizes, fees and refunds
5	Phone numbers (8–15 digits with optional + and separators)	`phonenumbr`	Premium-rate and call-back numbers
6	Remaining numbers	`numbr`	Keeps the signal without inflating the vocabulary
7	Other non-letter characters	removed	Leaves clean, readable word tokens

- **Health and Safety (user safety):** a false positive can suppress a legitimate OTP or an important mail, so high precision is prioritised on the SMS channel; no live user data is collected — only public, anonymised research corpora are processed.
- **Manufacturability (reproducibility):** pinned dependencies, fixed random seeds, a single-command training pipeline and a 13-check automated smoke-test suite guarantee that any examiner can rebuild the system.
- **Ethical and Social:** the scam-keyword lexicon is a plain, inspectable list in `src/config.py`; global SHAP summaries are used to audit the models for dataset bias; both corpora are credited to their original authors.

3.3. Analysis and Feature finalization subject to constraints

The constraint set directly shaped the final feature list. The interpretability requirement eliminated stemming and deep contextual encoders, and dictated the readable-placeholder normalisation: a token such as `httpaddr` can appear verbatim inside a LIME highlight, whereas a deleted URL cannot appear anywhere. The commodity-CPU constraint fixed the representation at TF–IDF plus engineered statistics, giving compact feature spaces of 3,015 dimensions for SMS and 5,015 for e-mail. The user-safety constraint resolved the

imbalance-handling question in favour of balanced class weights and `scale_pos_weight` rather than synthetic oversampling, so that every training example remains a real, explainable message. Finally, the reproducibility constraint required every artefact — processed datasets, trained detectors, metrics, figures — to be regenerated by one command.

3.4. Design Flow

Two alternative designs were carried forward for detailed comparison.

3.4.1. Design I: Fine-tuned transformer classifier

A single compact transformer (e.g. DistilBERT) fine-tuned jointly on both corpora, with attention maps and gradient-based attributions offered as explanations. Expected accuracy is at or marginally above the classical state of the art, and one model serves both channels. However, fine-tuning requires GPU time, inference is an order of magnitude slower on CPU, and — decisively — attention weights are an unfaithful basis for explanation, so the design fails the core transparency requirement of the client.

3.4.2. Design II: Hybrid classical ensemble with a dual XAI layer

Each modality receives its own feature space and its own trained ensemble of three heterogeneous learners — Logistic Regression, Random Forest and XGBoost — combined in probability space with weights proportional to each model's cross-validated F1. LIME explains the complete ensemble locally by re-scoring word-dropout perturbations through the whole pipeline; exact TreeSHAP explains the XGBoost component locally and globally. Training is CPU-only and completes in about two minutes. The block diagram of this alternative is shown in Figure 3.1.

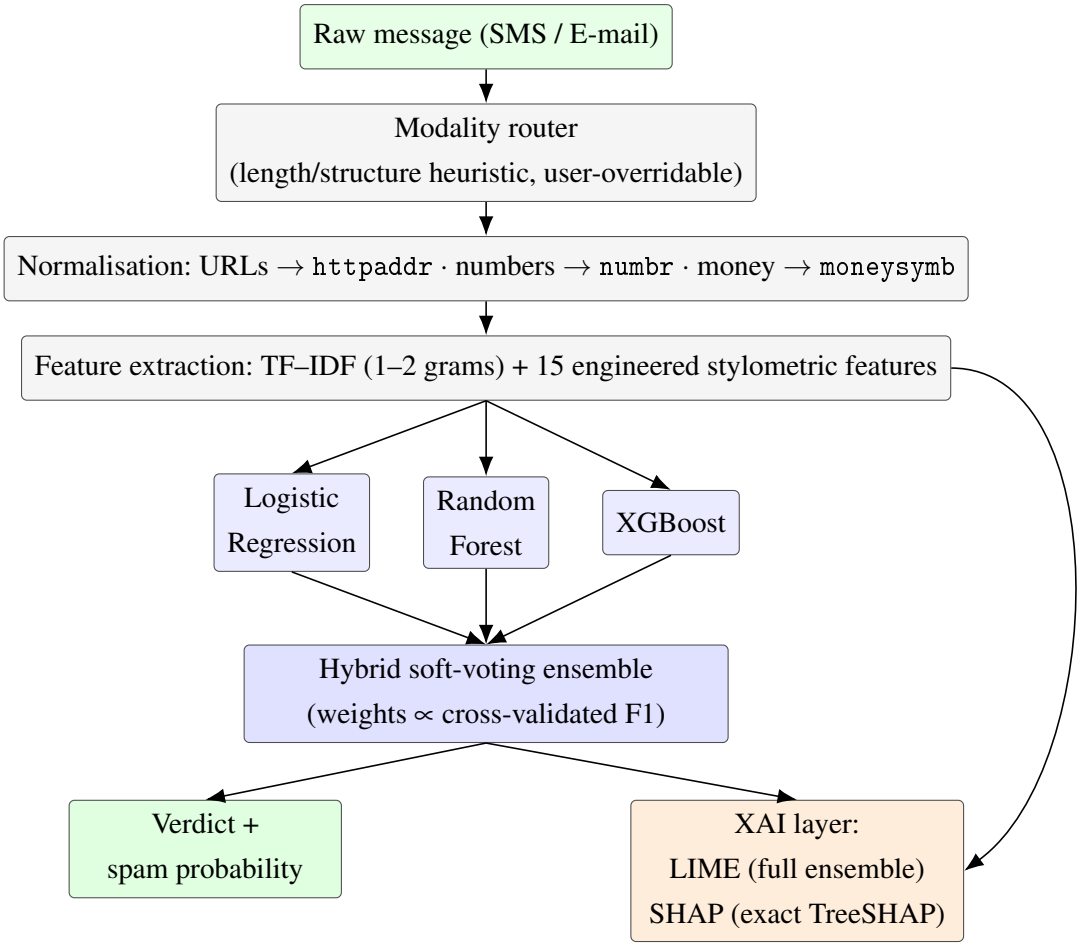


Figure 3.1: Block diagram of the proposed interpretable multi-modal detector (Design II)

3.5. Design selection

The two alternatives were scored against the constraint set, as shown in Table 3.2. Design II was selected: it reaches the accuracy band of the transformer on these corpora while satisfying every constraint the transformer violates, and its tree-based component admits *exact* Shapley-value explanations rather than approximations.

Table 3.2: Comparison and selection of the design alternatives

Criterion	**Design I (transformer)**	**Design II (hybrid + XAI)**
Expected / measured accuracy	98–99% (reported)	98.25–98.55% (measured)
Explanation fidelity	Low (attention is unfaithful)	High (model-agnostic LIME; exact TreeSHAP)
Training hardware / time	GPU; tens of minutes–hours	CPU; ≈2 minutes
Reproducibility for examiners	Moderate	High (single command + tests)
Meets full constraint set	No	Yes

3.6. Implementation plan/methodology

The selected design is realised by the pipeline of Figure 3.2. Text is vectorised with sublinear TF–IDF using the smoothed inverse document frequency of scikit-learn,

$$w_{t,d} \;=\; \bigl(1+\ln \mathrm{tf}_{t,d}\bigr)\cdot\left(\ln\frac{1+N}{1+\mathrm{df}_t}+1\right), \tag{3.1}$$

after which every document vector is scaled to unit length, and the fifteen engineered statistics are appended to the vector. Each base learner is trained with imbalance handling, and its 5-fold cross-validated F1 on the training split determines its voice in the ensemble:

$$p(\text{spam}\mid x) \;=\; \sum_{m=1}^{3} w_m\, p_m(\text{spam}\mid x), \qquad w_m \;=\; \frac{F1_m^{\mathrm{CV}}}{\sum_{k=1}^{3} F1_k^{\mathrm{CV}}}, \tag{3.2}$$

so a stronger learner gets a proportionally louder vote. The held-out test split is touched exactly once, for the final evaluation of Chapter 4.

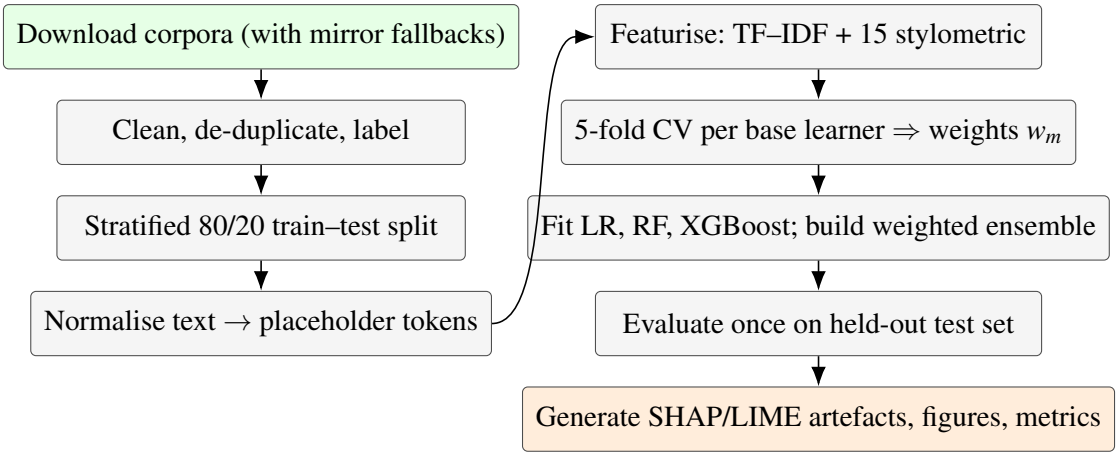


Figure 3.2: Implementation flowchart of the training and evaluation pipeline

3.6.1. Hyper-parameter configuration

All hyper-parameters are kept in one configuration module (`src/config.py`), so every experiment can be reproduced exactly. Table 3.3 lists the values used for both specialists. The learners use standard, moderately regularised settings rather than aggressively tuned ones, because the aim of the project is a transparent and reproducible system; the three learners nevertheless reach near-identical cross-validated F1 scores (Appendix-3).

Table 3.3: Hyper-parameters and experimental settings

Component	**Settings**
TF–IDF	SMS: 3,000 terms, minimum document frequency 2; e-mail: 5,000 terms, minimum document frequency 3; uni- and bi-grams; sublinear term frequency; accents stripped
Logistic Regression	`liblinear` solver; $C = 1.0$; up to 2,000 iterations; balanced class weights
Random Forest	250 trees; minimum of 2 samples per leaf; balanced class weights; all CPU cores
XGBoost	300 trees; maximum depth 6; learning rate 0.1; row subsampling 0.9; column subsampling 0.8; histogram tree method; log-loss; scale_pos_weight $= N_{\text{ham}}/N_{\text{spam}}$ on the training split (7.03 for SMS, 1.07 for e-mail)
Ensemble	Weights proportional to mean 5-fold cross-validated F1; decision threshold 0.5
Protocol	Stratified 80/20 split; 5 cross-validation folds; random seed 42; e-mail sample of 12,000 messages
Explanations	LIME: 1,000 perturbations, 10 words per explanation; global SHAP: 800 held-out messages per modality

3.6.2. Training and inference procedures

The complete training procedure of one specialist is given as Algorithm 1.

Algorithm 1: Training one channel specialist

Input: cleaned, de-duplicated corpus $D = \{(x_i, y_i)\}$ of one channel

Output: featuriser, three base learners, ensemble weights, evaluation report

1. Split D into 80% training and 20% test data, stratified by label (seed 42).
2. Fit the featuriser on the training texts only: learn the TF–IDF vocabulary of the normalised text and the maximum of each stylometric feature.
3. Compute scale_pos_weight $= N_{\text{ham}}/N_{\text{spam}}$ on the training split.
4. For each learner $m \in \{\text{LR}, \text{RF}, \text{XGB}\}$, estimate F1_m^{CV} by stratified 5-fold cross-validation on the training split.
5. Fit each learner on the full training split.
6. Set $w_m = \text{F1}_m^{\text{CV}} / \sum_k \text{F1}_k^{\text{CV}}$ and form the weighted soft-voting ensemble.
7. Evaluate the three learners and the ensemble once on the test split; save the metrics, the figures and the detector bundle.

3.6.3. Modality router

The router decides which specialist scores a message when the user has not chosen one. It is deliberately simple and fully transparent: a message is treated as e-mail if it is longer than 280 characters, contains a line break, or contains one of the words *subject*, *dear*, *regards*, *sincerely*, *unsubscribe* or *attached*; every other message is treated as SMS. The 280-character limit lies well above the 160 characters of a single SMS segment, and the cue words are typical of e-mail salutations, sign-offs and mailing lists. Because a rule of this kind can be wrong, the application lets the user override it, and its accuracy is measured in Section 4.1.5.

CHAPTER 4.

RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of solution

Modern engineering tools were used throughout:

- **Analysis:** Python (pandas, NumPy) and a Jupyter research notebook for exploratory data analysis and sampling studies.
- **Design and implementation:** scikit-learn (Logistic Regression, Random Forest, TF–IDF), XGBoost, and the SHAP and LIME libraries, organised as a modular `src/` package.
- **Report preparation:** LaTeX for this document; matplotlib with a shared publication style for all generated figures.
- **Project management and communication:** Git for version control and a shared Kanban board for task tracking.
- **Testing and validation:** a 13-check automated end-to-end smoke-test suite (including a simulated application session) and a strict held-out evaluation protocol.

Figure 4.1 shows the Detector tab of the Streamlit application after a message has been analysed: the routing decision, the verdict with the ensemble spam probability, and the vote of each base learner with its weight.

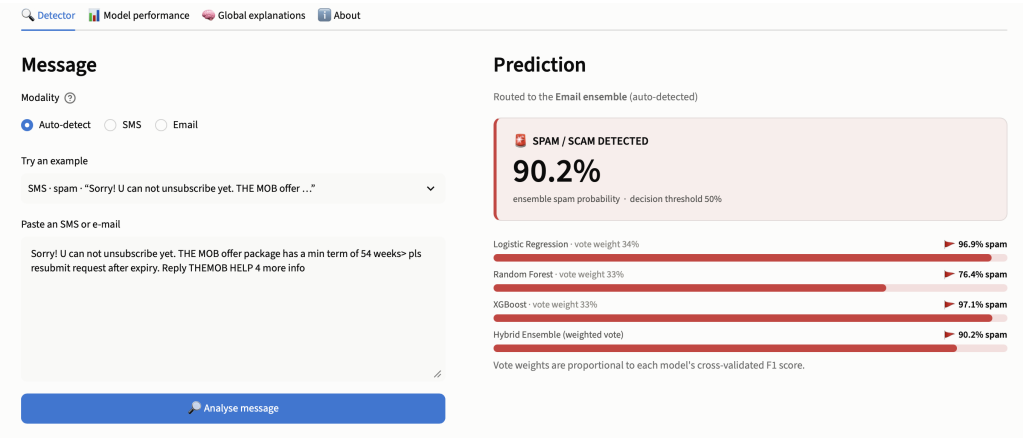


Figure 4.1: Detector tab of the Streamlit application after analysing a message

4.1.1. Experimental setup

Two public research corpora were used; their statistics after cleaning and de-duplication are given in Table 4.1 and visualised in Figure 4.2. Each corpus received an 80/20 stratified train–test split; base-learner weights were computed by 5-fold cross-validation on the training split only, and the test split was used exactly once. Performance is reported with accuracy, precision $P = \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, recall $R = \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, their harmonic mean F1, and ROC-AUC.

Table 4.1: Corpora used for training and evaluation (after cleaning)

Modality	Corpus	Messages	Spam share	Median words
SMS	UCI SMS Spam Collection (Almeida & Gómez Hidalgo, 2011)	5,158	12.4%	12
E-mail	Enron-Spam, stratified subsample (Metsis et al., 2006)	12,000	48.3%	155

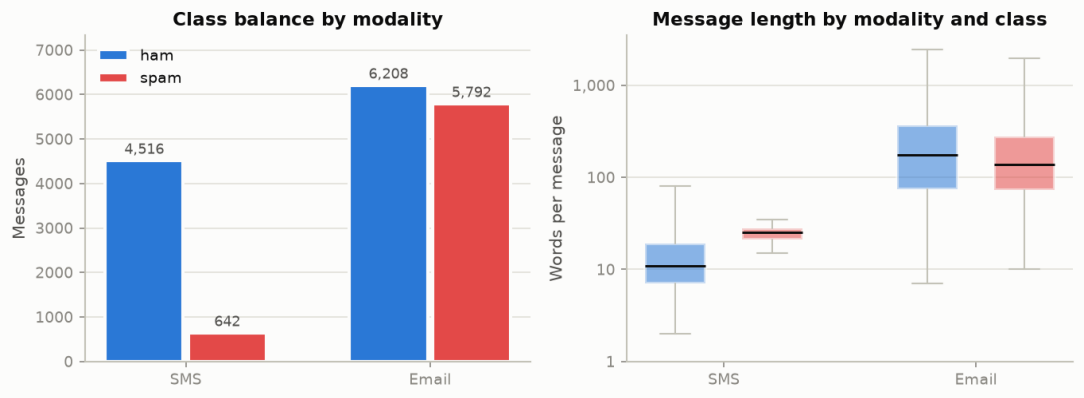


Figure 4.2: Class balance per modality and message-length distributions

The two modalities differ radically in both imbalance (12% versus 48% spam) and length (12 versus 155 median words), which is precisely why each is served by its own feature space and its own trained ensemble.

Exploratory analysis of the vocabulary. Before any model was trained, the corpora were inspected for the words that best separate the two classes. For every token present in at least eight messages after normalisation, the log-ratio $\ln\big(P(\text{token} \mid \text{spam})/P(\text{token} \mid \text{ham})\big)$ of its presence probabilities was computed (positive values indicate spam). On SMS the strongest spam indicators are prize and premium-rate vocabulary (*claim*, *prize*, *guaranteed*, *awarded*, *ppm*, *pobox*, *ringtone*) together with the placeholder `phonenumbr`, while ham is marked by informal chat (*lor*, *lol*, *cos*, *later*) and by *lt* and *gt*, the remains of the HTML-escaped

placeholder `<#>` that replaces masked numbers in the corpus and appears in 197 legitimate messages but in no spam. On e-mail the spam side is dominated by pharmaceutical and software-piracy vocabulary (*viagra*, *cialis*, *meds*, *photoshop*, *macromedia*), but the ham side is dominated by Enron-specific tokens: the company name *enron* itself (log-ratio -8.39), the surname *kaminski*, the abbreviations *hpl* and *ena*, and energy-trading jargon (*hourahead*, *mmbtu*, *westdesk*). This was the first sign of the dataset bias that the explanation layer later confirmed (Section 4.1.6.4).

Statistical protocol. Because all systems score above 97%, differences between them are small and must be checked for significance. Uncertainty is quantified with 95% percentile bootstrap confidence intervals over 2,000 resamples of each test set (Efron and Tibshirani, 1993). The ensemble is compared with each base learner on the same test messages with the exact McNemar test on discordant predictions (McNemar, 1947) and with DeLong's test for correlated ROC curves (DeLong et al., 1988); p-values are corrected for the three comparisons per modality with Holm's method (Holm, 1979). The Matthews correlation coefficient (MCC), the area under the precision–recall curve (PR-AUC), the false-positive rate (FPR) and the Brier score of the predicted probabilities complement the headline metrics.

4.1.2. Classification results

Tables 4.2 and 4.3 report the complete held-out results for all four systems on each modality; Figure 4.3 compares them graphically and Figure 4.4 shows the ROC curves.

Table 4.2: SMS — held-out test set (1,032 messages)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV F1
Logistic Regression	0.9855	0.9520	0.9297	0.9407	0.9941	0.9554 ± 0.020
Random Forest	0.9845	0.9516	0.9219	0.9365	0.9929	0.9558 ± 0.017
XGBoost	0.9845	0.9667	0.9062	0.9355	0.9941	0.9586 ± 0.015
Hybrid Ensemble	**0.9855**	**0.9669**	**0.9141**	**0.9398**	**0.9941**	—

Table 4.3: E-mail — held-out test set (2,400 messages)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV F1
Logistic Regression	0.9833	0.9753	0.9905	0.9829	0.9979	0.9797 ± 0.004
Random Forest	0.9746	0.9629	0.9853	0.9740	0.9973	0.9669 ± 0.004
XGBoost	0.9750	0.9645	0.9845	0.9744	0.9969	0.9732 ± 0.003
Hybrid Ensemble	**0.9825**	**0.9705**	**0.9940**	**0.9821**	**0.9983**	—

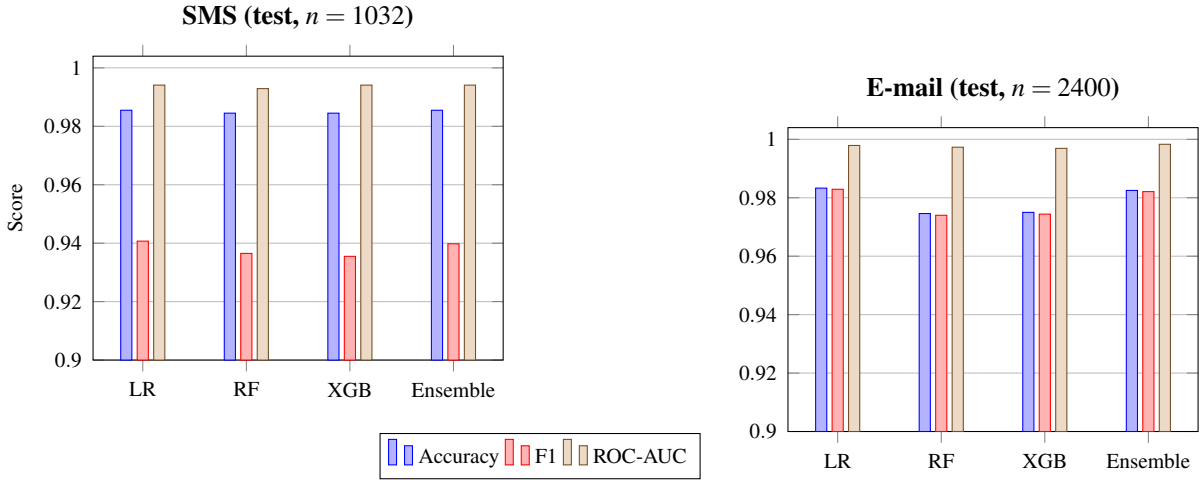


Figure 4.3: Test-set metrics per model and modality (vertical axis truncated at 0.90 to resolve the differences)

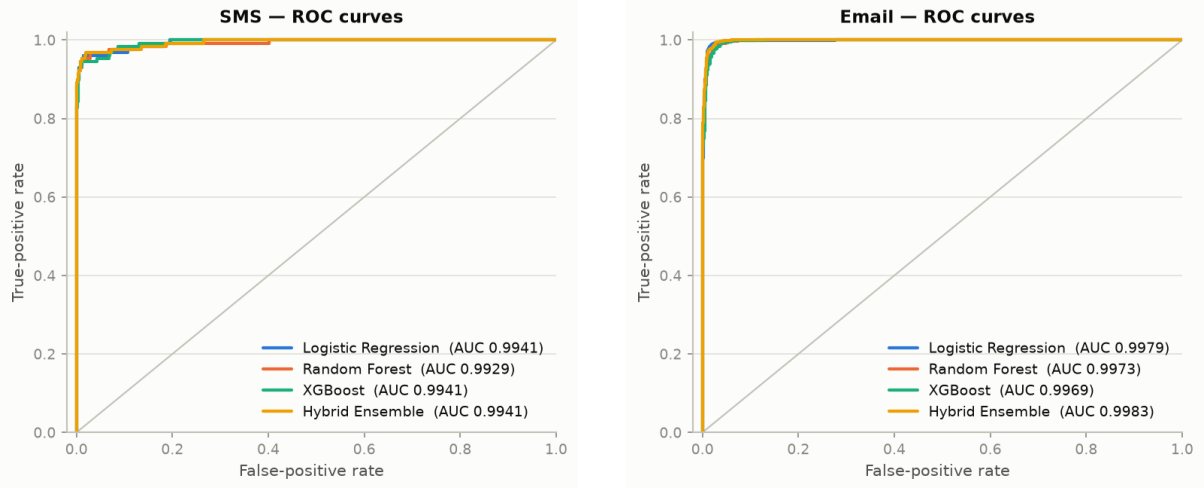


Figure 4.4: ROC curves for all four systems: SMS (left) and e-mail (right)

On ROC-AUC the hybrid ensemble ties the best base learners on SMS (0.9941, identical to four decimal places) and wins outright on e-mail (0.9983), where it also posts near-perfect recall (0.9940) at 0.9705 precision. On SMS it pairs the highest precision of any system (0.9669)

with the joint-highest accuracy (0.9855, tied with Logistic Regression) — exactly the profile the user-safety constraint asked for, since on that channel a false alarm means a suppressed legitimate message.

Figure 4.5 shows the four threshold metrics side by side for each modality. The plots make the scale of the differences visible: on SMS all four systems lie within 0.001 of each other in accuracy, and the largest spread, in recall (0.906–0.930), corresponds to only three of the 128 spam messages in the test set.

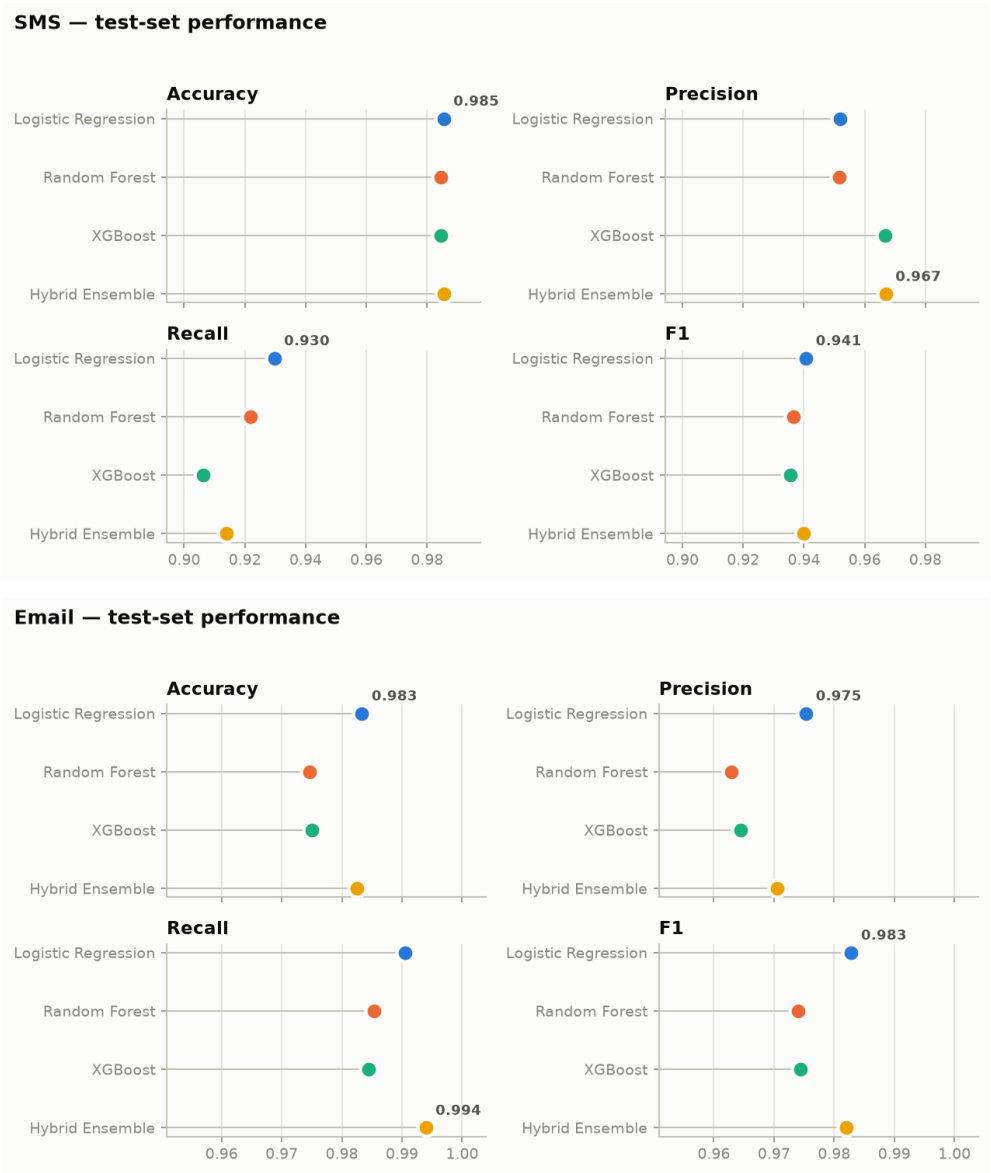


Figure 4.5: Test-set accuracy, precision, recall and F1 of the four systems: SMS (top) and e-mail (bottom)

Additional metrics. Table 4.4 reports metrics that are more informative than accuracy under class imbalance. The ensemble has the best PR-AUC on both modalities (0.9790 on SMS, 0.9980 on e-mail) and the best Brier score on SMS (0.0130), so its probabilities are both well ranked and accurate there; on e-mail, XGBoost and Logistic Regression have slightly lower Brier scores. Its MCC (0.9320 on SMS, 0.9652 on e-mail) is within 0.002 of the best learner

on both modalities, and its SMS false-positive rate of 0.44% equals the best, that of XGBoost.

Table 4.4: Additional metrics on the held-out test sets (lower Brier score and FPR are better)

Modality	Model	MCC	PR-AUC	Brier	FPR
SMS	Logistic Regression	0.9325	0.9768	0.0148	0.0066
	Random Forest	0.9278	0.9770	0.0184	0.0066
	XGBoost	0.9273	0.9739	0.0135	0.0044
	Hybrid Ensemble	0.9320	0.9790	0.0130	0.0044
E-mail	Logistic Regression	0.9668	0.9976	0.0210	0.0233
	Random Forest	0.9494	0.9970	0.0380	0.0354
	XGBoost	0.9502	0.9965	0.0189	0.0338
	Hybrid Ensemble	0.9652	0.9980	0.0213	0.0282

Confidence intervals. Table 4.5 gives the 95% bootstrap confidence intervals of the ensemble. They are narrow on e-mail (about ± 0.5 percentage points of accuracy) but wide on SMS for the spam-class metrics: with only 128 spam messages in the SMS test set, the recall interval spans 0.861–0.960. Differences of a few thousandths between systems on SMS are therefore well inside the sampling uncertainty.

Table 4.5: Hybrid ensemble: point estimates and 95% bootstrap confidence intervals (2,000 resamples of the test set)

Metric	SMS	95% CI	E-mail	95% CI
Accuracy	0.9855	[0.9777, 0.9922]	0.9825	[0.9771, 0.9875]
Precision	0.9669	[0.9286, 0.9925]	0.9705	[0.9607, 0.9799]
Recall	0.9141	[0.8605, 0.9600]	0.9940	[0.9894, 0.9982]
F1	0.9398	[0.9068, 0.9669]	0.9821	[0.9766, 0.9873]
MCC	0.9320	[0.8954, 0.9625]	0.9652	[0.9547, 0.9751]
ROC-AUC	0.9941	[0.9882, 0.9987]	0.9983	[0.9974, 0.9990]
PR-AUC	0.9790	[0.9610, 0.9934]	0.9980	[0.9971, 0.9989]

4.1.3. Statistical significance of the differences

Table 4.6 lists the paired tests. On SMS none of the differences is significant: the ensemble and each base learner disagree on at most eight of the 1,032 test messages, and the ROC-AUC differences are below 0.0013. On e-mail the picture is clearer. The ensemble is significantly better than both tree models — it classifies 24 messages correctly that Random Forest gets wrong, against 5 in the other direction, and 22 against 4 for XGBoost (Holm-corrected McNemar $p = 0.002$) — and its ROC-AUC is significantly higher (DeLong $p \leq 0.003$). Against Logistic Regression, however, the counts are balanced (11 against 13) and neither test

is significant. Combining a linear model with two tree ensembles therefore lifts the tree models to the level of Logistic Regression and improves the ranking metrics, but on these corpora it does not surpass a well-regularised linear model on TF–IDF features.

Table 4.6: Paired significance tests of the hybrid ensemble against each base learner (b / c: messages that only the ensemble / only the base learner classifies correctly; p-values Holm-corrected)

Modality	Comparison	b / c	McNemar p	ΔAUC	DeLong p
SMS	Ensemble vs. Logistic Regression	4 / 4	1.000	+0.0000	1.000
SMS	Ensemble vs. Random Forest	2 / 1	1.000	+0.0012	0.754
SMS	Ensemble vs. XGBoost	1 / 0	1.000	-0.0000	1.000
E-mail	Ensemble vs. Logistic Regression	11 / 13	0.839	+0.0004	0.208
E-mail	Ensemble vs. Random Forest	24 / 5	0.002	+0.0009	0.003
E-mail	Ensemble vs. XGBoost	22 / 4	0.002	+0.0014	<0.001

4.1.4. Ensemble behaviour and error analysis

Figure 4.6 shows the ensemble confusion matrices derived from the held-out evaluations. The SMS ensemble mislabels only 4 of 904 legitimate messages; the e-mail ensemble misses just 7 of 1,158 spam messages. The two matrices are deliberately asymmetric: the SMS specialist errs on the side of precision, the e-mail specialist on the side of recall, matching the relative costs of a wrong decision on each channel.

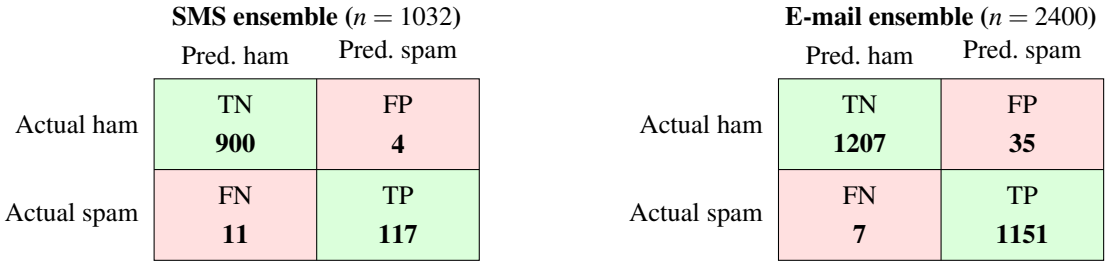


Figure 4.6: Confusion matrices of the hybrid ensembles on the held-out test sets

The confusion matrices of all four systems (Figure 4.7) show where the ensemble's advantage comes from. On e-mail it has the lowest false-negative count of all systems (7, against 11 for Logistic Regression, 17 for Random Forest and 18 for XGBoost), with a false-positive count (35) between that of Logistic Regression (29) and the tree models (42 and 44). On SMS it

matches the four false positives of XGBoost while missing one spam message fewer (11 against 12).

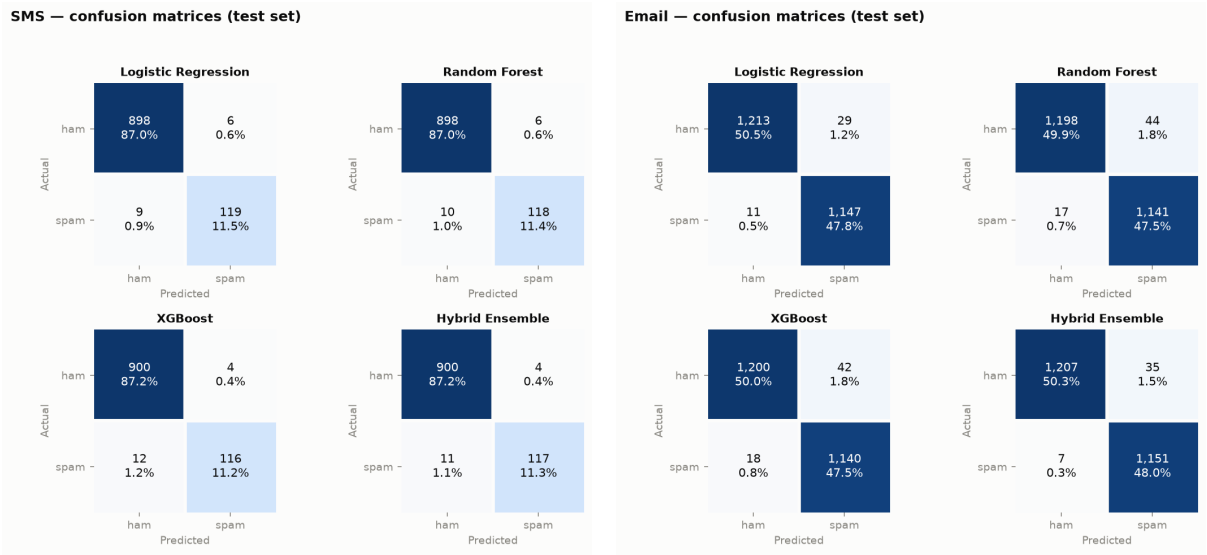


Figure 4.7: Confusion matrices of all four systems on the held-out test sets: SMS (left) and e-mail (right)

Error analysis. Table 4.7 lists representative misclassified test messages with the ensemble's spam probability. The errors are instructive. The SMS false positives share surface traits of spam: prices and a deposit, informal talk about money, and a genuine bank security notice whose vocabulary (*sensitive information*, *passwords*, *PIN*) is exactly that of phishing. The SMS false negatives are mostly joke or quiz messages written in a conversational style, with few of the digits, prices and links on which the stylometric features rely. On e-mail, the false positives include short forwarded notes and external newsletters, and none of the 35 contains the token *enron*; the missed spam includes an advance-fee fraud written in plain business language and a one-line lure to open an attachment, both of which lack the vocabulary that characterises most of the spam class.

4.1.5. Routing and cross-channel transfer

The per-modality results assume that every message reaches the correct specialist. Table 4.8 tests that assumption on the pooled test sets (3,432 messages). The specialists do not transfer: the SMS model is no better than chance on e-mail (ROC-AUC 0.53, MCC 0.04), and the e-mail model labels 88% of legitimate SMS as spam, so channel-specific training is essential. The heuristic router sends 96.4% of SMS and 85.1% of e-mails to the right specialist; the misrouted e-mails are short and contain none of the cue words, and the SMS specialist classifies only 42.3% of them correctly. As a result, end-to-end accuracy on the pooled test sets falls from 98.34% with oracle routing to 92.16% with automatic routing. The classifiers are accurate; the router is the weakest component of the deployed system, which is why the application lets the user override it. Figure 4.1 shows such a case: an SMS containing the word *unsubscribe* is

Table 4.7: Representative misclassified test messages (ensemble spam probability; threshold 0.5)

Error	Message (truncated)	p(spam)
SMS FP	Please protect yourself from e-threats. SIB never asks for sensitive information like Passwords,ATM/SMS PIN …	0.816
SMS FP	Hey… Great deal… Farm tour 9am to 5pm $95/pax, $50 deposit by 16 May	0.778
SMS FN	Latest News! Police station toilet stolen, cops have nothing to go on!	0.065
SMS FN	In The Simpsons Movie released in July 2007 name the band that died at the start of the film? A-Green Day, …	0.202
E-mail FP	the negotiation skills company , inc . newsletter aug 2001 fighting fires without burning bridges …	0.824
E-mail FP	fw : very symbolic of our current struggle	0.675
E-mail FN	please read urgently dear sir / madam , i am contacting you from asia to discuss a business deal of us $ 18 …	0.308
E-mail FN	re : your details please read the attached file .	0.323

routed to the e-mail ensemble.

Table 4.8: Routing and cross-channel transfer of the ensembles (pooled test set of 3,432 messages)

Setting	Accuracy	F1	MCC	AUC	FPR
Oracle routing (true channel), pooled	0.9834	0.9780	0.9648	0.9978	0.0182
Automatic routing, pooled	0.9216	0.8921	0.8317	0.9687	0.0443
SMS model applied to e-mail test set	0.5208	0.5287	0.0442	0.5290	0.5129
E-mail model applied to SMS test set	0.2209	0.2299	0.0594	0.6707	0.8805

4.1.6. Explainability results

4.1.6.1. A verdict, explained (local scope)

The following scam SMS, never seen in training, is used as a live exhibit:

> "URGENT! You have WON a £2,000 cash prize GUARANTEED. Call 09061701461 now or visit www.claim-your-prize.com to claim!"

All three base learners vote spam — Logistic Regression 1.000, Random Forest 0.987, XGBoost 1.000 — giving a weighted ensemble probability of **0.996 spam**. LIME re-scores approximately 1,000 word-dropout perturbations of the message through the entire pipeline

and fits a local linear surrogate; the resulting per-word contributions are plotted in Figure 4.8 alongside the SHAP waterfall of the same prediction. The premium-rate phone number is the single strongest signal, followed by the money amount and the classic scam vocabulary — a justification a non-expert can check at a glance.

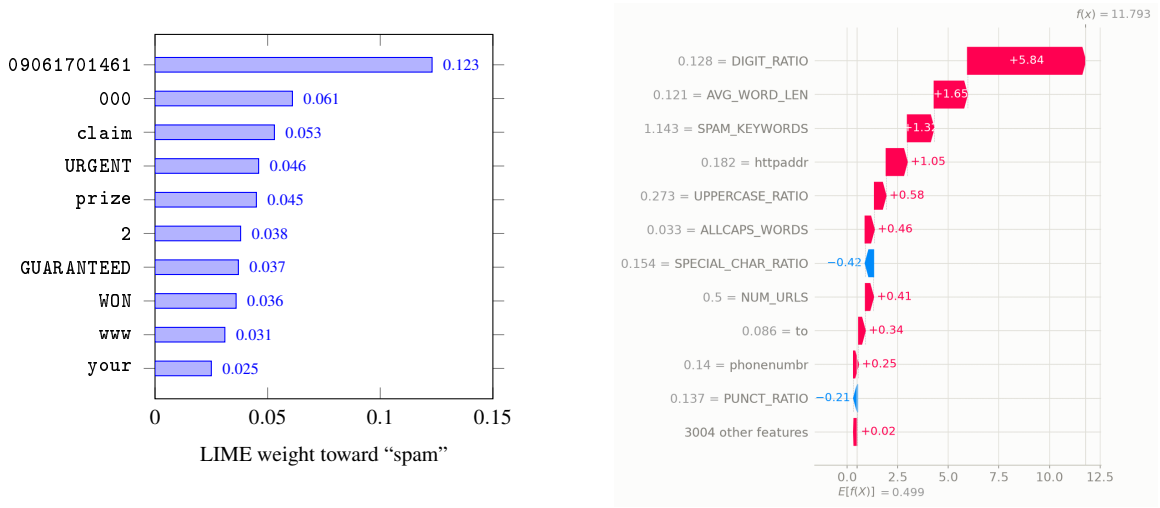


Figure 4.8: Local explanations of the exhibited SMS: LIME word attributions over the full ensemble (left) and SHAP waterfall of the XGBoost component (right)

Figure 4.9 applies LIME to two real spam messages from the test sets, each the highest-scoring among the first 200 spam messages of its test set. For the SMS, the premium-rate number, the PO-box code and *URGENT* carry most of the weight; for the e-mail, the link and promotional words (*online*, *save*, *free*) push towards spam, while *thanks* — typical of legitimate correspondence — pushes the other way, showing that LIME reports evidence in both directions.

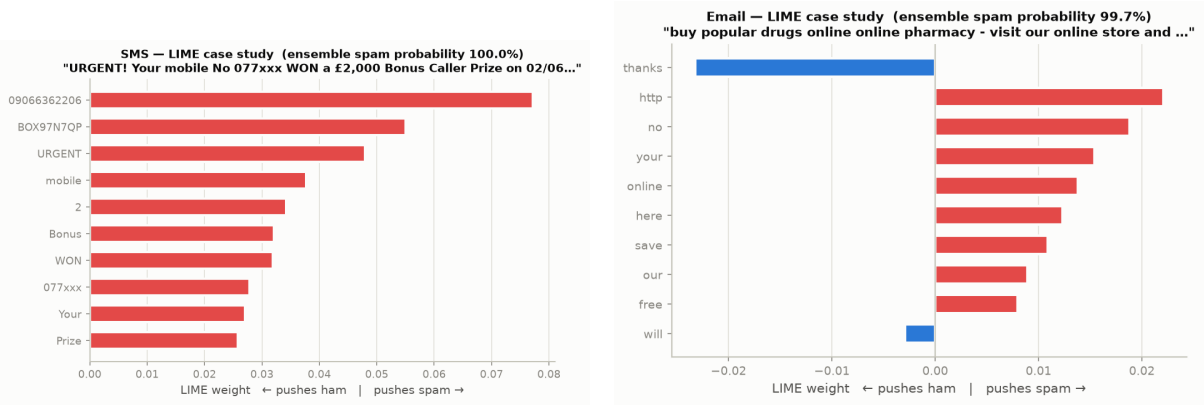


Figure 4.9: LIME explanations of two held-out spam messages over the full ensemble: SMS (left) and e-mail (right); red bars push towards spam, blue towards ham

4.1.6.2. Are the explanations faithful?

An explanation is useful only if it reflects what the model actually relies on. This was tested with a deletion experiment: for 60 SMS and 25 e-mail spam messages that the ensemble detected correctly, the top-k words ranked by LIME ($k = 1, 3, 5$) were removed and the drop in the ensemble's spam probability was recorded, and compared with removing k randomly chosen words (averaged over ten draws). Table 4.9 shows that LIME's words matter far more than random ones. On SMS, removing the single top word lowers the spam probability by 0.106 on average, 8.6 times more than a random word, and removing the top five lowers it by 0.455 and flips 52% of the verdicts to ham. On e-mail the drops are smaller in absolute terms, because long messages contain redundant evidence, but still about 20 times larger than for random deletions.

Table 4.9: Deletion test of LIME faithfulness: mean drop in the ensemble spam probability after removing the top-k LIME words / k random words

Modality	$k = 1$	$k = 3$	$k = 5$	**Flipped** ($k = 5$)
SMS ($n = 60$)	0.106 / 0.012	0.291 / 0.043	0.455 / 0.081	52%
E-mail ($n = 25$)	0.043 / 0.002	0.133 / 0.005	0.198 / 0.010	24%

4.1.6.3. What the models learned (global scope)

Global SHAP summaries were computed over 800 test messages per modality (Figure 4.10). The key finding is that *stylometry rivals vocabulary*: on SMS the five top-ranked features are all engineered stylometric statistics — `DIGIT_RATIO`, `AVG_WORD_LEN`, `SPAM_KEYWORDS`, `UPPERCASE_RATIO` and `CHAR_COUNT` — and the fifteen engineered features together carry about three-quarters (74.5%) of the total attribution, although they are only 15 of the 3,015 features. A scam is betrayed by its *shape* as much as by its words.

All SHAP values in this chapter are computed with XGBoost's native, exact TreeSHAP on the same sparse feature matrix that the model scores. This matters: XGBoost treats entries that are absent from a sparse matrix as missing values, so explaining a densified copy of the input would describe a different function — in this pipeline, densification changes the XGBoost verdict for 729 of the 1,032 SMS test messages.

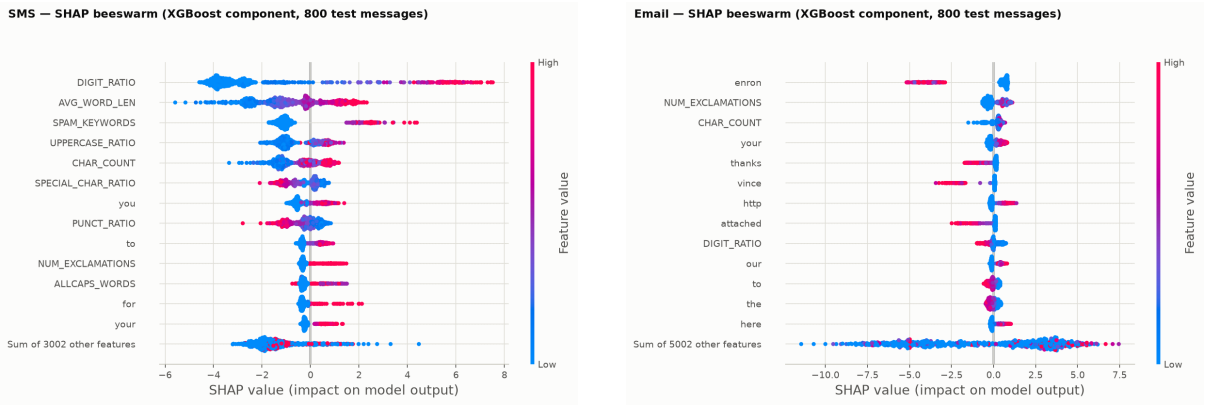


Figure 4.10: Global SHAP beeswarm summaries: SMS (left) and e-mail (right); UPPER-CASE rows are engineered stylometric features, lower-case rows are words

Figure 4.11 condenses the beeswarms into the mean absolute attribution of each feature, and Tables 4.10 and 4.11 list the ten most important features with the direction of their effect. On SMS, eight of the ten are stylometric: a high share of digits, longer words, scam-lexicon hits, capital letters and a longer message all push towards spam, while a high share of punctuation and special characters, typical of informal chat, pushes towards ham. On e-mail the ranking is led by a single word, *enron*, whose mean attribution (1.327) is three times that of the next feature; the first name *vince* and correspondence words such as *thanks* and *attached* also push towards ham, whereas exclamation marks, links and the words *your* and *our* push towards spam.

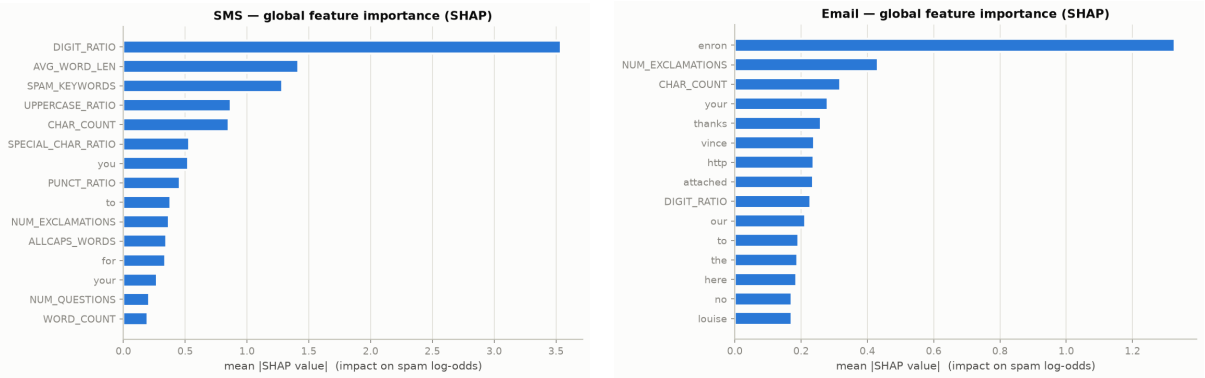


Figure 4.11: Global feature importance (mean |SHAP|, exact TreeSHAP on the XGBoost component, 800 held-out messages): SMS (left) and e-mail (right)

4.1.6.4. XAI as a debugging tool: a dataset bias, caught

On the e-mail model, the global SHAP ranking exposed the token `enron` as a strong *ham* indicator: the corpus's legitimate mail comes entirely from the mailboxes of Enron employees, so the model partly learned the dataset rather than the concept. This bias is invisible in every accuracy table in this chapter and obvious in one beeswarm plot — the clearest argument this project makes for shipping explanations together with predictions.

Table 4.10: SMS: the ten most important features by mean |SHAP| (exact TreeSHAP, XGBoost component, 800 held-out messages); the last column shows where high values of the feature push the prediction

Rank	Feature	Type	Mean $\lvert\phi\rvert$	Pushes towards
1	DIGIT_RATIO	Stylometric	3.539	spam
2	AVG_WORD_LEN	Stylometric	1.414	spam
3	SPAM_KEYWORDS	Stylometric	1.285	spam
4	UPPERCASE_RATIO	Stylometric	0.867	spam
5	CHAR_COUNT	Stylometric	0.851	spam
6	SPECIAL_CHAR_RATIO	Stylometric	0.532	ham
7	you	Word / placeholder	0.523	spam
8	PUNCT_RATIO	Stylometric	0.457	ham
9	to	Word / placeholder	0.379	spam
10	NUM_EXCLAMATIONS	Stylometric	0.368	spam

Table 4.11: E-mail: the ten most important features by mean |SHAP| (exact TreeSHAP, XGBoost component, 800 held-out messages); the last column shows where high values of the feature push the prediction

Rank	Feature	Type	Mean $\lvert\phi\rvert$	Pushes towards
1	enron	Word / placeholder	1.327	ham
2	NUM_EXCLAMATIONS	Stylometric	0.432	spam
3	CHAR_COUNT	Stylometric	0.318	spam
4	your	Word / placeholder	0.281	spam
5	thanks	Word / placeholder	0.260	ham
6	vince	Word / placeholder	0.239	ham
7	http	Word / placeholder	0.238	spam
8	attached	Word / placeholder	0.236	ham
9	DIGIT_RATIO	Stylometric	0.229	ham
10	our	Word / placeholder	0.213	spam

Quantifying the shortcut. The token occurs in 44.4% of the legitimate training e-mails and 42.1% of the legitimate test e-mails, but in none of the spam messages in either split, so it is a perfect one-sided signal for ham. The exact attributions confirm that the model relies on it: its mean TreeSHAP contribution is -3.95 log-odds when the token is present, against $+0.66$ when it is absent. Two probes measured the practical consequences (Table 4.12). Deleting the token from the 523 legitimate test e-mails that contain it flips only 11 verdicts, because legitimate mail contains other ham evidence. Appending the single word *enron* to spam, however, flips 409 of the 1,151 spam messages the ensemble had detected (35.5%) to ham. The learners differ sharply: XGBoost is evaded on 91.7% of the messages, Random Forest on 25.3% and Logistic Regression on only 4.5%. The heterogeneous ensemble is therefore far more robust than its boosted member, though not immune — a concrete argument for combining learners with different inductive biases.

Table 4.12: Shortcut probe on the e-mail test set: appending the word "enron" to detected spam, and deleting it from legitimate mail ($\overline{\Delta p}$: mean change in spam probability)

Model	Spam flipped to ham	$\overline{\Delta p}$	Ham flipped to spam	$\overline{\Delta p}$
Logistic Regression	52 / 1147 (4.5%)	-0.066	2 / 523	+0.032
Random Forest	289 / 1141 (25.3%)	-0.301	10 / 523	+0.053
XGBoost	1045 / 1140 (91.7%)	-0.763	19 / 523	+0.052
Hybrid Ensemble	409 / 1151 (35.5%)	-0.376	11 / 523	+0.046

4.1.7. Computational cost

Table 4.13 summarises the cost of training and of explanation. Training a specialist, including 5-fold cross-validation of all three learners, takes 14 seconds for SMS and 77 seconds for e-mail on a CPU. A single verdict takes about 15 ms, an exact TreeSHAP attribution under a millisecond and a LIME explanation with 1,000 perturbations 0.08–0.21 s, so every prediction can be explained while the user waits. Batched scoring processes about 15,000 SMS or 2,000 e-mails per second.

Table 4.13: Computational cost on a 10-core ARM64 CPU without a GPU (median times)

Modality	Training (s)	Verdict (ms)	Batch (msg/s)	SHAP (ms)	LIME (ms)
SMS	14.3	14.7	14,897	0.86	76
E-mail	76.8	14.8	1,967	0.75	210

4.1.8. Validation on realistic unseen messages

The public corpora are more than a decade old, so the system was also tested on 28 realistic messages written for this project in the style of current scams and everyday messages: lottery

and cash-back offers, electricity disconnection threats, parcel and KYC scams, part-time job offers, account suspension and password-expiry phishing, alongside ordinary personal texts, meeting notes and university e-mails. None of these messages appears in either corpus. With automatic routing, all 28 reached the intended specialist and 26 were classified correctly; Table 4.14 lists a representative selection together with both errors. The first error is a genuine one-time password that receives a spam probability of 52.2%: OTP messages are full of digits, exactly like scams, which is why the SMS specialist was tuned for precision. The second is an invoice-fraud e-mail (38.9%) that contains none of the vocabulary typical of spam; Logistic Regression flags it (65%) but the tree models do not.

The shortcut of Section 4.1.6.4 can also be demonstrated live. A password-expiry phishing e-mail receives a spam probability of 86.6%; signing the same message as "IT Support, Enron North America" lowers it to 42.3% and flips the verdict to ham, with XGBoost dropping from 86.8% to 3.1% while Logistic Regression still reports 80.4%.

Table 4.14: Validation on realistic messages written for this project (not taken from either corpus); spam probability from the routed ensemble

Channel	Message	True	Predicted	Correct
SMS	Congratulations! Your mobile number has won Rs 25,00,000 in our annual lucky draw. To claim your prize call 9876543210 now and share your bank details.	spam	spam (99.0%)	Yes
SMS	Your electricity connection will be disconnected tonight at 9:30 PM due to an unpaid bill. Call our officer on 9123456780 immediately to avoid disconnection.	spam	spam (91.6%)	Yes
SMS	Hey, are we still meeting for lunch at 1pm? I might be 10 min late.	ham	ham (7.9%)	Yes
SMS	Mom, I reached the hostel safely. Will call you after dinner.	ham	ham (3.4%)	Yes
E-mail	Subject: Password expiry notice. Your mailbox password expires today. Click http://mail-verify-portal.com to keep your account active and avoid losing your messages. IT Support	spam	spam (86.6%)	Yes

Channel	Message	True	Predicted	Correct
E-mail	Subject: Your account has been suspended. Dear customer, we detected unusual activity. Verify your account details at http://secure-account-check.com within 24 hours or your account will be closed. Customer Service	spam	spam (85.2%)	Yes
E-mail	Subject: Q3 planning meeting notes. Hi team, attached are the notes from yesterday's planning session. Please review the action items assigned to you before Friday's follow-up. Thanks, Sarah	ham	ham (14.0%)	Yes
SMS	Your OTP for login is 482913. It is valid for 10 minutes. Do not share it with anyone.	ham	spam (52.2%)	**No**
E-mail	Subject: Invoice payment overdue. Please find the attached invoice and make the payment of 2,450 dollars today using the link http://pay-invoice-online.net to avoid late fees. Accounts Department	spam	ham (38.9%)	**No**

4.1.9. Interpretation and data validation

Cross-validation dispersion is small (F1 standard deviation between ± 0.003 and ± 0.020 across all six base learners), indicating stable models rather than lucky splits, and the derived ensemble weights are near-uniform (Appendix-3), so the ensemble's gains come from the *diversity* of the three learners rather than from one dominant model. The 13-check automated smoke-test suite exercises the entire system — data loading, preprocessing, both detectors, LIME, SHAP and a simulated application session — and passes end-to-end; the complete pipeline reproduces from a clean checkout in about three minutes on a commodity CPU, with training itself taking about two minutes. The dataset downloader validates its sources and falls back to mirrors automatically, and the modality router can be overridden by the user in the application, so a misrouted message can always be re-scored by the other specialist.

Table 4.15 lists the thirteen automated checks and the result of the final run.

Table 4.15: Automated end-to-end smoke tests (final run: all checks passed)

No.	Check	Result
1	SMS detector flags a scam SMS as spam	Pass ($p = 0.996$)
2	SMS detector passes an ordinary SMS as ham	Pass ($p = 0.079$)
3	All three SMS base learners return a vote	Pass
4	LIME returns word weights for the scam SMS	Pass
5	SHAP returns attributions for the scam SMS	Pass
6	E-mail detector flags a phishing e-mail as spam	Pass ($p = 0.945$)
7	E-mail detector passes a meeting-notes e-mail as ham	Pass ($p = 0.140$)
8	All three e-mail base learners return a vote	Pass
9	LIME returns word weights for the phishing e-mail	Pass
10	SHAP returns attributions for the phishing e-mail	Pass
11	Application renders without an exception	Pass
12	Clicking *Analyse* renders without an exception	Pass
13	The verdict is shown to the user	Pass

CHAPTER 5.

CONCLUSION AND FUTURE WORK

5.1. Conclusion

Every objective set in Section 2.6 was met. The hybrid ensemble reached 98.55% accuracy on SMS and 98.25% on e-mail against a target of 97%, and it matched (to four decimal places) or beat every base learner on ROC-AUC on both modalities (0.9941 on SMS; an outright best 0.9983 on e-mail). Every single prediction is delivered with a local LIME explanation over the full ensemble and a SHAP attribution of the boosted-tree component, and global beeswarm and importance summaries describe what each model has learned. The complete pipeline — corpus download, training, evaluation and figure generation — reproduces from a clean checkout in about three minutes on a commodity CPU and is verified by a 13-check automated test suite, and the four-tab Streamlit application makes the system operable by a non-expert.

The most instructive result, however, was not a number. The global SHAP analysis exposed the token `enron` acting as a shortcut for legitimate mail — a genuine dataset bias in the e-mail corpus that deviated from the naive expectation that a 98%-accurate model has learned the concept of "spam". The bias is a property of the data, not a fault of the pipeline, but it would have shipped silently inside any black-box deployment. Catching it required the explanation layer, which is precisely the thesis this project set out to demonstrate: interpretability is not a presentation feature bolted onto a classifier, it is a debugging and auditing instrument that changes what the engineer can see.

The additional validation reported in Chapter 4 strengthens these conclusions. Paired tests show that the ensemble is significantly better than both tree models on e-mail and statistically level with Logistic Regression; a deletion test confirms that LIME's explanations point at the words on which the decision really depends; the `enron` shortcut was quantified, with a single inserted word evading the ensemble on 35.5% of spam e-mails and XGBoost alone on 91.7%; and the heuristic router was identified as the weakest link of the deployed system. Of 28 realistic messages written for this project, 26 were classified correctly.

5.2. Future work

- Extend the framework to multilingual and code-mixed corpora, since the present datasets are English-only and date from 2002–2012.
- Perform cross-domain evaluation (train on one corpus, test on another) to quantify and then mitigate the dataset bias exposed by SHAP.

- Add probability calibration so that the reported spam probabilities can be thresholded meaningfully by downstream policy.
- Harden the models against adversarially obfuscated spelling (e.g. leet-speak substitutions) and measure robustness explicitly.
- Add counterfactual explanations — "what is the smallest edit that flips this verdict?" — as a third, complementary XAI technique.
- Benchmark a fine-tuned transformer under the same explanation harness to quantify the accuracy–interpretability trade-off empirically.
- Replace the heuristic router with a learned channel classifier, since automatic routing currently lowers pooled accuracy from 98.3% to 92.2%.
- Mask organisation-identity tokens such as `enron` during training and measure how accuracy and robustness change.
- Compute the SHAP waterfall in the application directly on the sparse feature matrix, as done for the figures in this report, so that the interactive explanations match them exactly.

APPENDIX

Appendix-1: Software environment

S. No.	Component	Version	Role in the framework
1	Python	3.10 or newer (tested on 3.12 and 3.14)	Implementation language of the entire pipeline
2	scikit-learn	1.4 or newer	Logistic Regression, Random Forest, TF–IDF, metrics, cross-validation
3	XGBoost	2.0 or newer	Gradient-boosted tree base learner
4	SHAP	0.45 or newer	Exact TreeSHAP local and global attributions
5	LIME	0.2.0.1 or newer	Model-agnostic local explanations of the full ensemble
6	pandas / NumPy	2.2 / 1.26 or newer	Data handling and numerical computation
7	matplotlib	3.8 or newer	All generated figures (shared publication style)
8	Streamlit	1.36 or newer	Four-tab interactive demonstration application
9	Git	–	Version control and collaboration

Appendix-2: Engineered stylometric features (complete set)

S. No.	Feature	Description
1	CHAR_COUNT	Length of the message in characters
2	WORD_COUNT	Number of words
3	AVG_WORD_LEN	Average word length
4	DIGIT_RATIO	Share of characters that are digits
5	UPPERCASE_RATIO	Share of letters written in upper case
6	ALLCAPS_WORDS	Words written fully in capitals (e.g. ‘WINNER’)
7	PUNCT_RATIO	Share of punctuation characters
8	NUM_EXCLAMATIONS	Count of ‘!’
9	NUM_QUESTIONS	Count of ‘?’
10	NUM_CURRENCY	Currency symbols and money amounts
11	NUM_URLS	Links and web addresses detected
12	NUM_EMAILS	E-mail addresses detected
13	NUM_PHONES	Phone numbers detected
14	SPAM_KEYWORDS	Hits in the scam-keyword lexicon (Appendix-4)
15	SPECIAL_CHAR_RATIO	Share of non-alphanumeric characters

All fifteen features are computed on the raw message, before normalisation, and are defined in plain sight in `src/features.py`, together with the keyword lexicon in `src/config.py`, so that they can be audited without reading model internals.

Appendix-3: Derived ensemble weights (normalised CV F1)

Modality	Base learner	CV F1	Weight w_m	
SMS	Logistic Regression	0.9554	0.3329	$\sum w_m = 1$
	Random Forest	0.9558	0.3331	
	XGBoost	0.9586	0.3340	
E-mail	Logistic Regression	0.9797	0.3356	$\sum w_m = 1$
	Random Forest	0.9669	0.3311	
	XGBoost	0.9732	0.3333	

The near-uniform weights show that the three learners are comparably strong; the ensemble’s advantage therefore comes from their diversity (linear / bagging / boosting) rather than from any single dominant model.

Appendix-4: Scam-keyword lexicon

The `SPAM_KEYWORDS` feature counts whole-word, case-insensitive matches of the following 46 words, defined in `src/config.py`.

free	win	winner	won	winning
cash	prize	award	urgent	claim
congratulations	congrats	offer	click	subscribe
guaranteed	guarantee	credit	loan	lottery
million	bonus	discount	cheap	deal
limited	expires	expire	alert	verify
account	password	bank	refund	investment
viagra	pills	ringtone	voucher	gift
txt	sms	mobile	selected	reward
unsubscribe				